

PID 제어 정리 및 사용 방법

1. 개요

이 문서는 26-01-W11-TempPID-ADC 프로젝트의 PID 제어 로직 정리 문서로, UART 사용 절차, 튜닝 방법에 대하여 설명함. 또한 추가적으로 주의사항도 함께 기술함.

2. 현재 코드 기준 PID 구성

2.1 제어 대상

- 센서: TMP235 (ADC1 IN6, PA6)
- 구동 출력: TIM2 CH1 PWM (PA15)
- 제어 목적: 온도를 `Setpoint` 로 수렴시키도록 PWM 듀티(%) 조절함.

2.2 동작 모드

- `APP_MODE_MELODY`: 멜로디 출력 모드.
- `APP_MODE_PID`: PID 제어 모드.
- `APP_MODE_TEMP`: 온도 모니터 모드.
- `APP_MODE_IDLE`: 출력 정지 모드.

2.3 PID 파라미터(현재 코드값)

- `pid_setpoint_c = 25.0f` (기본 목표 온도)
- `pid_kp = 8.0f`
- `pid_ki = 0.12f`
- `pid_kd = 1.2f`
- 제어 주기: `PID_CONTROL_MS = 50ms`
- PWM 주파수: `PID_PWM_FREQ_HZ = 20Hz`

연관 코드 스니펫:

```
float pid_setpoint_c = PID_SETPOINT_DEFAULT_C;
float pid_kp = 8.0f;
float pid_ki = 0.12f;
float pid_kd = 1.2f;
float pid_integral = 0.0f;
float pid_prev_error = 0.0f;

#define PID_CONTROL_MS 50U
#define PID_PWM_FREQ_HZ 20U
```

참고: 실제 동작 기준은 항상 소스 코드(`Core/Src/main.c`)가 우선임.

3. PID 계산 로직 설명

PID 업데이트 함수의 핵심 순서는 아래와 같음.

1. 현재 온도 측정함.

- `pid_last_temp_c = TMP235_ReadTempC();`

2. 오차 계산함.

- `error = setpoint - measured_temp`

3. 적분항 업데이트 후 제한함.

- `pid_integral += error * dt`

- 제한 범위: `-100.0 ~ 100.0`

4. 미분항 계산함.

- `derivative = (error - pid_prev_error) / dt`

5. PID 출력 계산함.

- `output = Kp*error + Ki*integral + Kd*derivative`

6. 출력 제한함.

- PWM 듀티 범위를 `0.0 ~ 100.0%`로 클램프함.

7. PWM에 반영함.

- `PWM_SetDutyPercent(output)`

8. 이전 오차 저장함.

- `pid_prev_error = error`

연관 코드 스니펫:

```
static void PID_Update(void)
{
    float error;
    float derivative;
    float output;
    const float dt = ((float)PID_CONTROL_MS) / 1000.0f;

    pid_last_temp_c = TMP235_ReadTempC();
    error = pid_setpoint_c - pid_last_temp_c;

    pid_integral += error * dt;
    if (pid_integral > 100.0f) pid_integral = 100.0f;
    else if (pid_integral < -100.0f) pid_integral = -100.0f;

    derivative = (error - pid_prev_error) / dt;
    output = (pid_kp * error) + (pid_ki * pid_integral) + (pid_kd * derivative);

    if (output < 0.0f) output = 0.0f;
    if (output > 100.0f) output = 100.0f;

    pid_last_output = output;
    PWM_SetDutyPercent(output);
    pid_prev_error = error;
}
```

수식 형태

- 비례항: $P = K_p e(t)$
- 적분항: $I = K_i \int e(t) dt$
- 미분항: $D = K_d \frac{de(t)}{dt}$
- 출력: $u(t) = P + I + D$

이 프로젝트는 이산 시간 구현으로 샘플 주기 $dt = 0.05s$ 를 사용함.

4. 온도 측정 처리(TMP235 + ADC)

4.1 ADC 이동평균 필터

노이즈 감소를 위해 8샘플 이동평균 사용함.

- 샘플 수: `ADC_MOVING_AVG_SAMPLES = 8`
- 평균값을 `pid_last_adc_raw`로 저장.

연관 코드 스니펫:

```
static float TMP235_ReadTempC(void)
{
    static uint16_t adc_hist[ADC_MOVING_AVG_SAMPLES] = {0U};
    static uint32_t adc_sum = 0U;
    static uint8_t adc_idx = 0U;
    static uint8_t adc_count = 0U;
    uint16_t raw = ADC_ReadChannel(ADC_CHANNEL_6);
    uint16_t avg_raw;

    if (adc_count < ADC_MOVING_AVG_SAMPLES)
    {
        adc_sum += raw;
        adc_hist[adc_idx] = raw;
        adc_idx = (uint8_t)((adc_idx + 1U) % ADC_MOVING_AVG_SAMPLES);
        adc_count++;
        avg_raw = (uint16_t)(adc_sum / adc_count);
    }
    else
    {
        adc_sum -= adc_hist[adc_idx];
        adc_hist[adc_idx] = raw;
        adc_sum += raw;
        adc_idx = (uint8_t)((adc_idx + 1U) % ADC_MOVING_AVG_SAMPLES);
        avg_raw = (uint16_t)(adc_sum / ADC_MOVING_AVG_SAMPLES);
    }

    pid_last_adc_raw = avg_raw;
    return (((float)avg_raw * ADC_VREF_V) / ADC_MAX_COUNT) - TMP235_OFFSET_V / TMP235_SLOPE_V_PER_C;
}
```

4.2 전압/온도 변환

- 기준 전압: `ADC_VREF_V = 3.3V`
- ADC 최대 카운트: `4095`
- TMP235 보정값:

- `TMP235_OFFSET_V = 0.5V`
- `TMP235_SLOPE_V_PER_C = 0.01V/°C`

변환식:

$$V_{out} = \frac{ADC_{raw}}{4095} \times 3.3$$

$$T(^{\circ}C) = \frac{V_{out} - 0.5}{0.01}$$

5. UART 명령 사용 방법

시리얼 설정은 115200 / 8N1 임.

5.1 지원 명령

- `start` : Melody 모드 진입.
- `stop` : Idle 모드 진입.
- `temp` : Temp 모드 진입.
- `pid` : PID 제어 모드 진입.
- `sp <값>` : PID 목표온도 변경. (0~100)
- `coeff` : 현재 Kp/Ki/Kd 출력.
- `coeff kp <v>` : Kp 설정. (0~1000)
- `coeff ki <v>` : Ki 설정. (0~1000)
- `coeff kd <v>` : Kd 설정. (0~1000)
- `coeff all <kp> <ki> <kd>` : 계수 3개 일괄 설정.
- `cv=off` 또는 `cv off` : PID 리포트 텍스트 출력.
- `cv=on` 또는 `cv on` : PID 리포트 CSV 출력.
- `cv=0n` : `cv=on` 과 동일 처리.

연관 코드 스니펫:

```
else if ((strcmp(uart_cmd_buf, "coeff", 5) == 0) &&
        ((uart_cmd_buf[5] == '\0') || (uart_cmd_buf[5] == ' ') || (uart_cmd_buf[5] == '\t') ||
        (uart_cmd_buf[5] == '=')))
{
    if (has_value != 0U)
    {
        if (app_mode == APP_MODE_PID)
        {
            printf("Cannot change coeff in PID mode. Change mode first (stop/start/temp).\r\n");
        }
        /* PID 모드 밖에서만 계수 변경 */
    }
}
else if ((strcmp(uart_cmd_buf, "cv", 2) == 0) &&
        ((uart_cmd_buf[2] == '\0') || (uart_cmd_buf[2] == ' ') || (uart_cmd_buf[2] == '\t') ||
        (uart_cmd_buf[2] == '=')))
{
    if ((strcmp(p, "on") == 0) || (strcmp(p, "0n") == 0)) pid_cv_csv_mode = 1U;
```

```
else if (strcmp(p, "off") == 0) pid_cv_csv_mode = 0U;
}
```

제약:

- PID 모드에서는 계수 변경 차단.
- PID 모드 밖에서만 `coeff` 설정 허용.

5.2 사용 예시

```
stop
coeff all 9.0 0.20 1.40
pid
cv=on
sp 30.5
```

5.3 출력 로그 예

텍스트 모드(`cv=off`) 예:

```
PID RAW=1234 T=27.35C SP=30.5C OUT=41.2%
```

CSV 모드(`cv=on`) 예:

```
1234,27.35,30.5,41.2
```

필드 의미:

- `RAW`: 이동평균된 ADC Raw 값.
- `T`: 현재 온도.
- `SP`: 목표 온도(Setpoint).
- `OUT`: PID 계산 결과 PWM 듀티(%).

6. 실제 사용 절차(권장)

1. 전원 인가 후 시리얼 터미널 연결.
2. `temp` 명령으로 센서값 정상 여부 확인.
3. PID 사용 전 `stop` 또는 `temp` 상태로 유지.
4. `coeff` 또는 `coeff all` 로 계수 설정.
5. `pid` 명령으로 제어 모드 진입.
6. `sp 30.0` 처럼 목표 온도 설정.
7. 20Hz 로그를 보며 수렴 상태 확인.

연관 코드 스니펫:

```
if ((app_mode == APP_MODE_TEMP) && (HAL_GetTick() - temp_report_tick >= PID_CONTROL_MS))
{
    temp_report_tick = HAL_GetTick();
    temp_c = TMP235_ReadTempC();
    printf("TEMP RAW=%u T=%ld.%02ldC\r\n", ...);
}
```

```

}

if ((app_mode == APP_MODE_PID) && (HAL_GetTick() - pid_last_tick >= PID_CONTROL_MS))
{
    pid_last_tick = HAL_GetTick();
    PID_Update();
}

if ((app_mode == APP_MODE_PID) && (HAL_GetTick() - pid_report_tick >= PID_CONTROL_MS))
{
    pid_report_tick = HAL_GetTick();
    if (pid_cv_csv_mode != 0U) printf("%u,%ld.%02ld,%ld.%01ld,%ld.%01ld\r\n", ...);
    else printf("PID RAW=%u T=%ld.%02ldC SP=%ld.%01ldC OUT=%ld.%01ld%%\r\n", ...);
}

```

7. PID 튜닝 가이드(실무용 간단 버전)

7.1 기본 원칙

- K_p 를 먼저 조정해 반응성을 맞추는 방식.
- K_i 는 정상상태 오차 제거 용도로 천천히 증가.
- K_d 는 과도 진동/오버슈트 감쇠 용도.

7.2 추천 순서

1. $K_i=0$, $K_d=0$ 으로 두고 K_p 만 올려 반응 확인.
2. 목표 근처 정착 후 오차가 크면 K_i 를 소폭 증가.
3. 진동이 커지면 K_d 를 소량 증가.
4. 출력이 자주 0%/100%에 붙으면 K_p 또는 K_i 를 낮추는 것이 좋음.

7.3 증상별 점검

- 반응이 너무 느림: K_p 증가.
- 목표 근처 떨림: K_p 감소 또는 K_d 증가.
- 잔류 오차 큼: K_i 증가.
- 오버슈트 큼: K_p 감소 또는 K_d 증가.
- 출력 포화 지속: K_p , K_i 과대 가능성 있음.

8. 현재 코드의 보호/제약 사항

- 적분항 클램프: `pid_integral` 을 $-100 \sim 100$ 으로 제한함.
- 출력 클램프: PWM 듀티를 $0 \sim 100\%$ 로 제한함.
- Setpoint 입력 제한: $0 \sim 100^\circ\text{C}$ 범위만 허용함.
- PID/TEMP 리포트 주기: `50ms` (20Hz)임.

주의:

- 센서/히터 물리 지연이 큰 시스템에서는 빠른 튜닝 시 발산 가능성이 있음.

9. 빌드/검증 체크리스트

- 빌드 후 에러 없이 펌웨어 생성 확인.
- UART 명령 수신 정상 여부 확인.
- `temp` 에서 온도 값이 현실 범위인지 확인.
- `pid` 전환 후 `OUT` 값이 상황에 따라 변하는지 확인.
- `sp` 변경이 로그에 즉시 반영되는지 확인.
- `cv=on` 에서 CSV 형식 출력 확인.

10. 향후 개선 아이디어

- `sp` 램프(완만한 목표 변화) 적용.
- 미분 저역통과 필터 추가.
- 센서 단선/이상치 예외 처리 로직 추가.
- CSV 로그 기반 자동 튜닝 스크립트 연동.

11. 빠른 명령 요약

<code>start</code>	: 멜로디 모드
<code>stop</code>	: 출력 정지/계수 수정 준비
<code>temp</code>	: 온도 모니터 모드
<code>pid</code>	: PID 제어 모드
<code>sp 28.0</code>	: 목표온도 28.0C 설정
<code>coeff</code>	: 현재 계수 조회
<code>coeff kp 9.5</code>	: Kp 설정
<code>coeff ki 0.20</code>	: Ki 설정
<code>coeff kd 1.30</code>	: Kd 설정
<code>coeff all 9.5 0.20 1.30</code>	: 계수 일괄 설정
<code>cv=off</code>	: PID 텍스트 출력
<code>cv=on</code>	: PID CSV 출력